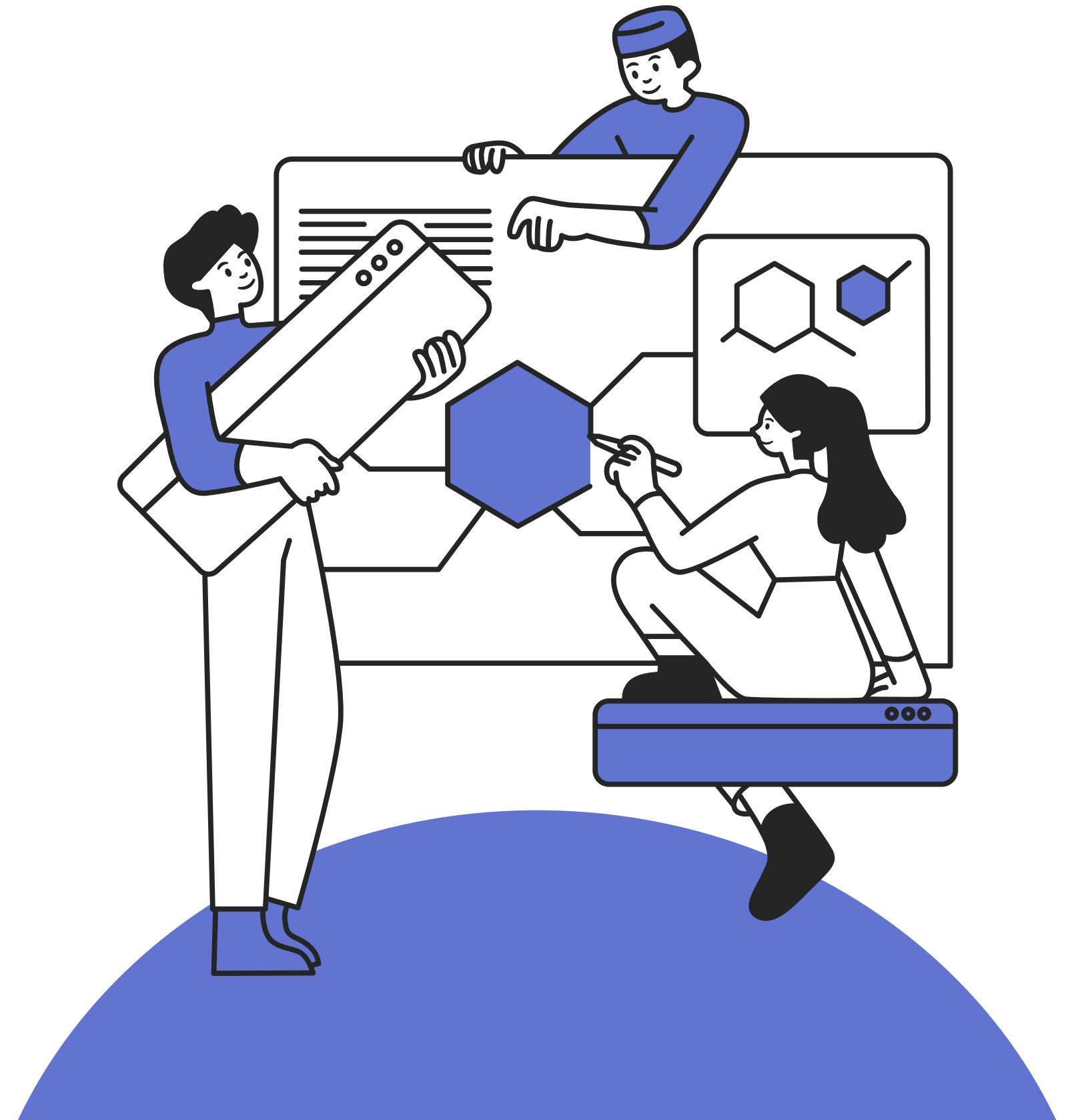


문자열 탐색

이예린T



CONTENTS

- 1 문자열 탐색
- 2 모듈별 탐구
- 3 문자열 탐색 정리

01

문자열 탐색

말 그대로

- 문자열 탐색(string searching): 문자열 안에 다른 문자열이 포함되어 있는지 검사하고, 만약 포함되어 있다면 그 위치를 찾아내는 것
- 코랩, VS Code, 웹브라우저, 워드 등 문자열 데이터를 다룰 때 기본 기능
- ‘IN’ 찾기
- ‘STRING’: 성공
- ‘SEJONG’: 실패
- 본문(텍스트): 탐색 대상 문자열, 검색되는 쪽의 문자열
- 패턴: 탐색어, 찾아내는 문자열
- 이동: 본문에서 탐색 위치 옮기기



Ctrl + F = Find

02

Brute Force법

무식한? 고지식한? 탐색 알고리즘

- 선형 탐색의 확장
- 본문의 첫 번째 문자와 패턴의 첫 번째 문자를 비교 → 본문 두 번째, 패턴 두 번째 문자 비교 → ...
- 불일치 문자 발견 시 본문 두 번째 문자와 패턴 첫 번째 문자 비교형 탐색의 확장
- 본문의 첫 번째 문자와 패턴의 첫 번째 문자를 비교 → 본문 두 번째, 패턴 두 번째 문자 비교 → ...
- 불일치 문자 발견 시 본문 두 번째 문자와 패턴 첫 번째 문자 비교
- 본문 길이 N , 패턴 길이 M 일 때 최악의 경우 $N \times M$ 번 비교

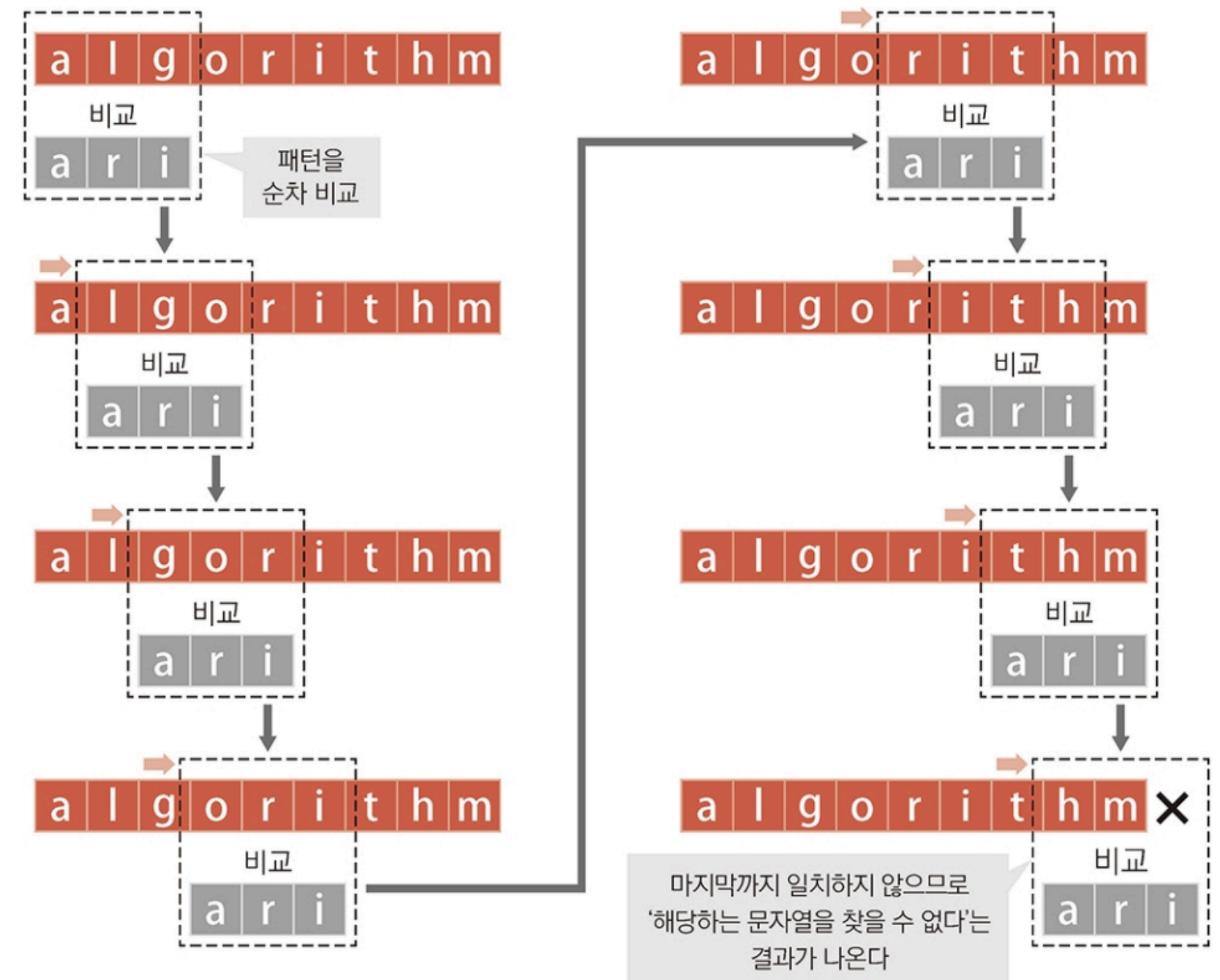


그림 4-3 완전 탐색 알고리즘을 이용한 문자열 검색

해시 함수를 써 보자

- 문자 1:1 비교가 아니라, 패턴의 해시값과 본문 내부 문자열의 해시값 비교
- S: 문자열, S[n]: 문자열 내 n번째 문자, m: 문자열 길이
- 해시 함수: 각 문자의 아스키코드 값 * 2의 제곱
- 매번 제곱과 총합 계산을 해야 하는가???

▶ abacaabb의 해시 값

$$= 97 * 2^7 + 98 * 2^6 + 97 * 2^5 + 99 * 2^4 + 97 * 2^3 + 97 * 2^2 + 98 * 2^1 + 98 * 2^0 = 24,834$$

비교 문자열

A	B	C	A	B
---	---	---	---	---

아스키코드

65	66	67	65	66
----	----	----	----	----

Hash값

$$65 * 2^4 + 66 * 2^3 + 67 * 2^2 + 65 * 2^1 + 66 * 2^0 = \mathbf{2032}$$

Step 1

parent 해시: 24824 ,pattern 해시: 24833

[illegible]

Step 2

parent 해시: 24913 , pattern 해시: 24833

[illegible]

Step 3

parent 해시: 24837 , pattern 해시: 24833

[illegible]

Step 4

parent 해시: 24939 , pattern 해시: 24833

[illegible]

Figure 1

Step 5

parent 해시: 24833 , pattern 해시: 24833

[illegible]

03

라빈 카프 알고리즘

해시 함수를 써 보자

- 해시 함수의 비용을 획기적으로 줄이는 방법 적용
- 계산을 마친 값을 재활용한다.
- 총 탐색 시간이 패턴 길이와 상관 없이 본문 길이에만 영향을 받음.
- 최초의 해시값을 구할 때는 기존의 해시 함수 사용
- 해시값이 일치했지만 충돌 상황일 수 있으므로, 해시값이 일치한 문자열을 찾은 후 각 문자가 패턴 문자와 일치하는지 한 차례 더 확인한다.

$$H[i] = S[i] * 2^{M-1} + S[i+1] * 2^{M-2} + \dots + S[i+M-2] * 2^1 + S[i+M-1] * 2^0$$

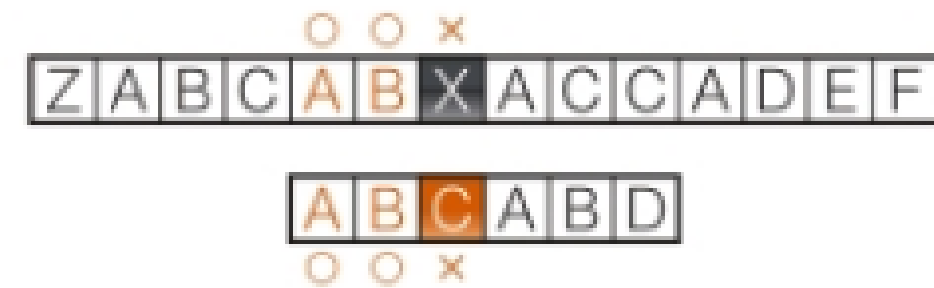
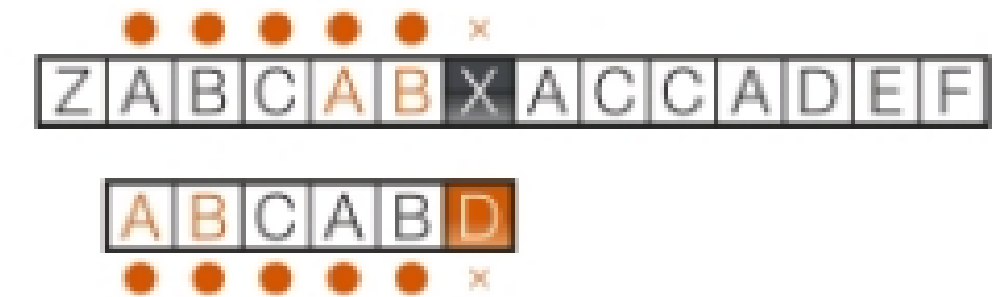
$$H[i+1] = 2 * (H[i] - C[i] * 2^{m-1}) + C[i+m+1] * 2^0$$

04

KMP 알고리즘

고지식하지만 조금 똑똑하게

- 비교할 필요 없는 부분은 지나치고 비교가 필요한 부분만 비교 수행
- 문자열을 한 차례 비교하고 나면, 다음 단계 탐색에서 사용할 수 있는 정보가 남는다는 개념 활용
- 검사했던 결과를 버리지 않고 효율적으로 활용
- 경계 테이블을 구성
- 본문과 패턴 사이 겹치는 부분을 찾아내 검사를 다시 시작할 위치를 구해서, 패턴의 이동 거리를 늘린다.
- 텍스트 스캔이 앞으로만 나아가고, 뒤로 돌아가는 일은 없다.



04

KMP 알고리즘

경계 테이블 만들기

ABCABD

ABCABD

ABCABD

ABCABD

ABCABD

ABCABD

ABCABD

ABCABD

A	B	C	A	B	D
-	0				

A	B	C	A	B	D
-	0	0			

A	B	C	A	B	D
-	0	0	1	2	

A	B	C	A	B	D
-	0	0	1	2	0

- 패턴 "ABCABD"를 두 줄로 겹쳐 놓고 아랫줄을 1칸 뒤로 옮긴다.
- 주황색 부분이 겹치지 않으므로 패턴을 다시 1칸 옮길 경우 1번째 문자부터 검사를 다시 시작해야 함을 알 수 있다.

- 그러므로 2번째 문자(B)의 다시 시작하는 값은 0이다.
- 이는 패턴의 1번째 문자(인덱스 0)부터 검사를 다시 시작한다는 의미이다.
- 다시 패턴을 1칸 뒤로 옮긴다.

- 문자가 일치하지 않으므로 표에서 3번째 문자(C)의 값을 0으로 한다.
- 다시 패턴을 1칸 뒤로 옮기면 "AB"가 일치한다.
- 따라서 표에서 4번째와 5번째 문자의 경우 다시 시작하는 값을 1, 2로 한다.

- 이어서 패턴을 2칸 뒤로 옮기면 문자가 일치하지 않는다.
- 표에서 패턴의 마지막 문자 'D'의 값을 0으로 한다.

1. 패턴의 4번째 문자 'A'까지 일치하면 패턴을 텍스트의 다음 'A'까지 한번에 옮긴 뒤 'A'를 건너뛰고 2번째 문자부터 검사할 수 있습니다(**e**).

2. 패턴의 4번째, 5번째 문자 'AB'까지 일치하면 패턴을 텍스트의 다음 'AB'까지 한번에 옮긴 뒤 'AB'를 건너뛰고 3번째 문자부터 검사할 수 있습니다(**f**).

04

KMP 알고리즘

탐색하기

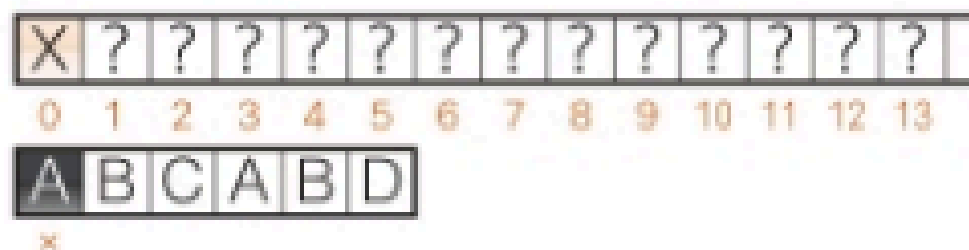
A A A A A A A B

A A A A B → 4번 일치 후 불일치

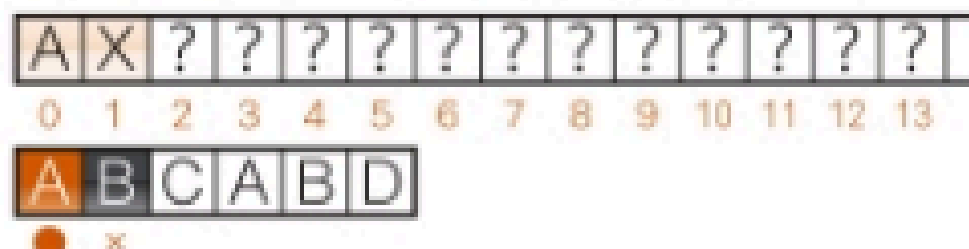
A A A A B → 3칸 점프,

AAA는 이미 일치 확인됨

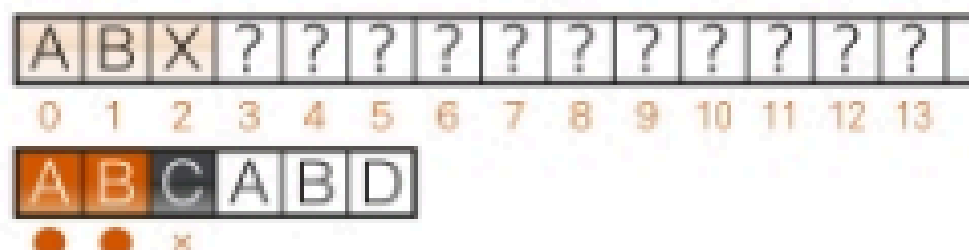
a 1번째 문자가 일치하지 않습니다.



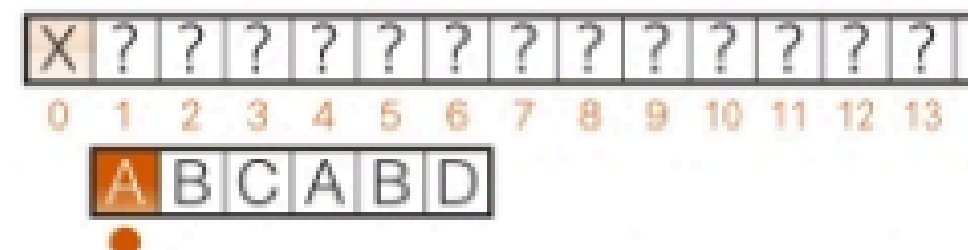
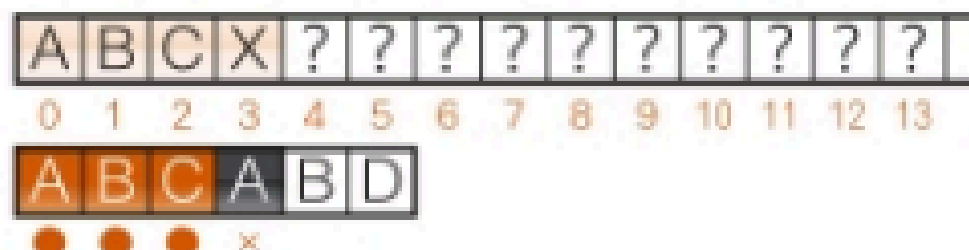
b 2번째 문자가 일치하지 않습니다.



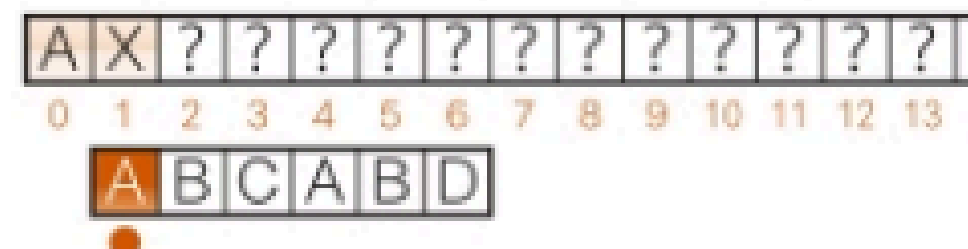
c 3번째 문자가 일치하지 않습니다.



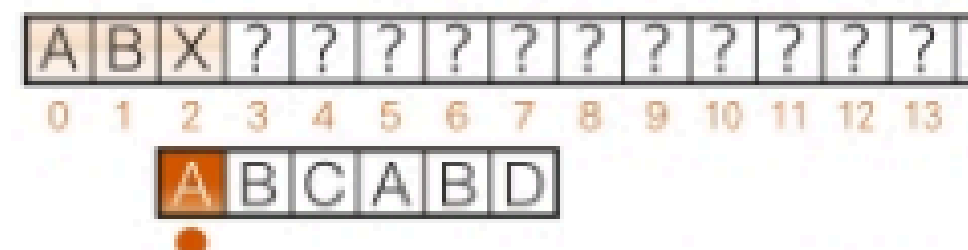
d 4번째 문자가 일치하지 않습니다.



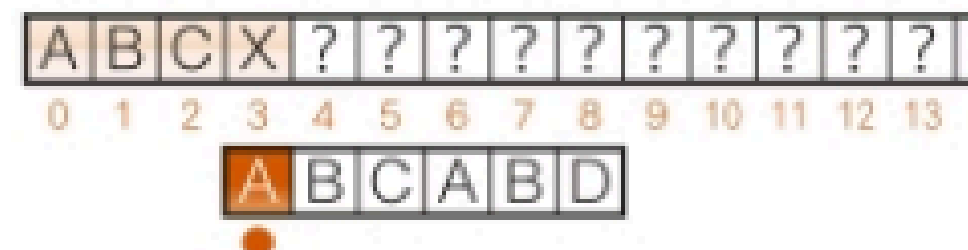
1번째 문자부터 검사를 다시 시작합니다.



1번째 문자부터 검사를 다시 시작합니다.



1번째 문자부터 검사를 다시 시작합니다.



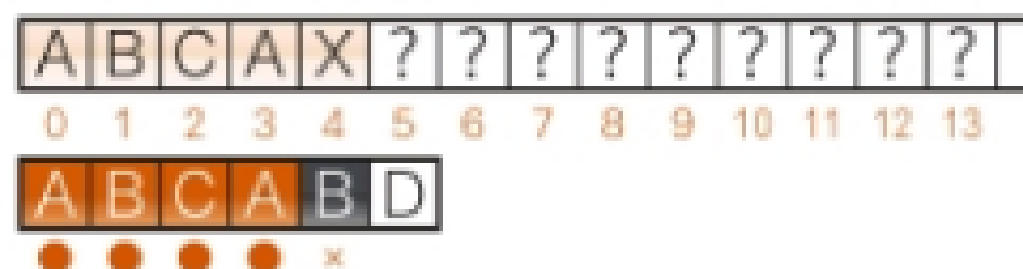
1번째 문자부터 검사를 다시 시작합니다.

04

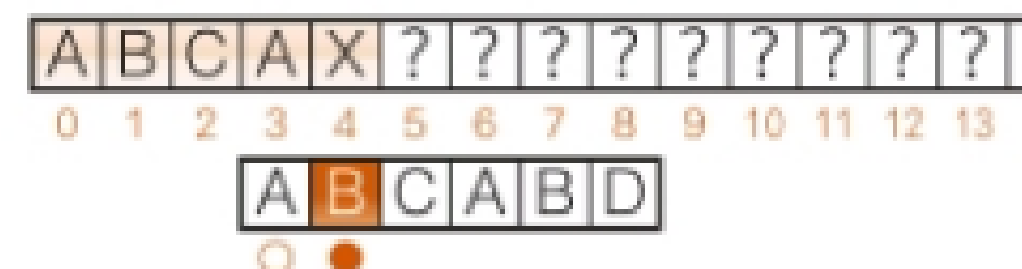
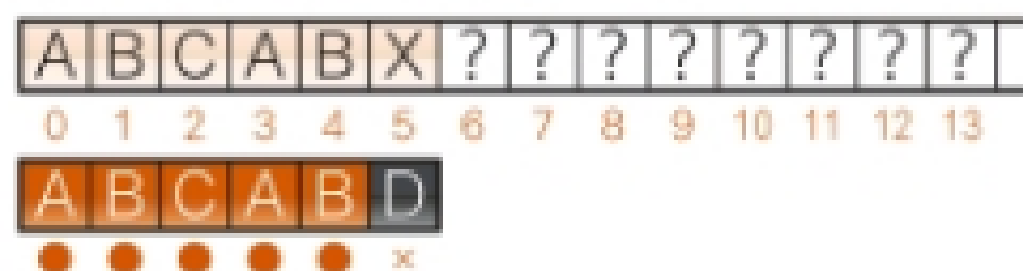
KMP 알고리즘

탐색하기

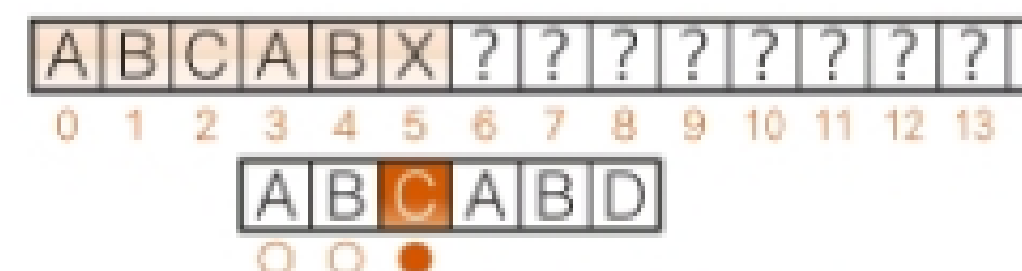
e 5번째 문자가 일치하지 않습니다.



f 6번째 문자가 일치하지 않습니다.



2번째 문자부터 검사를 다시 시작합니다.



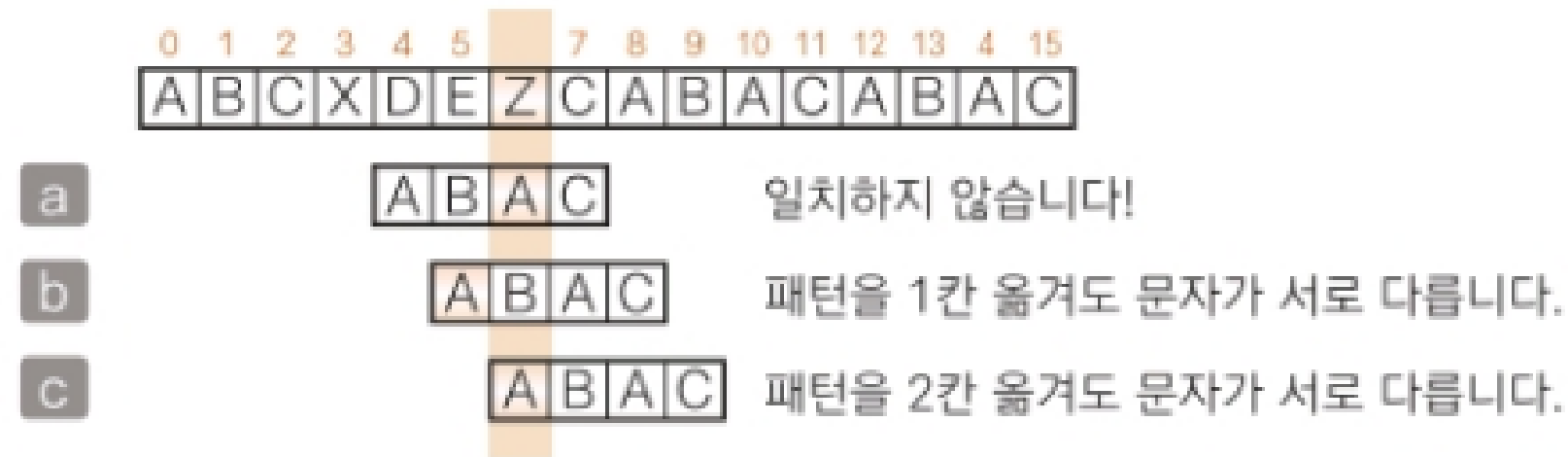
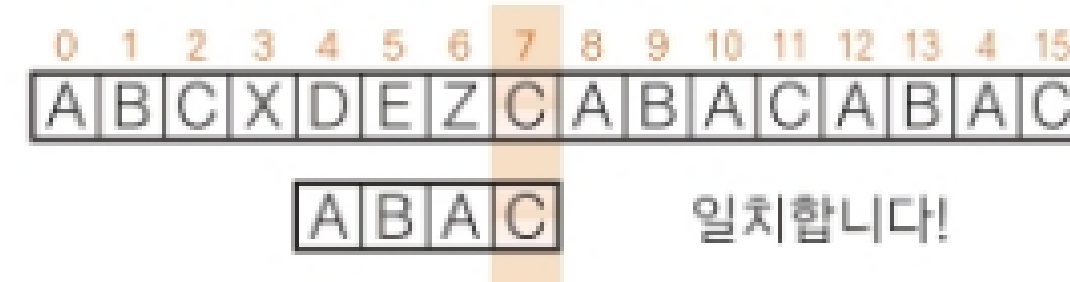
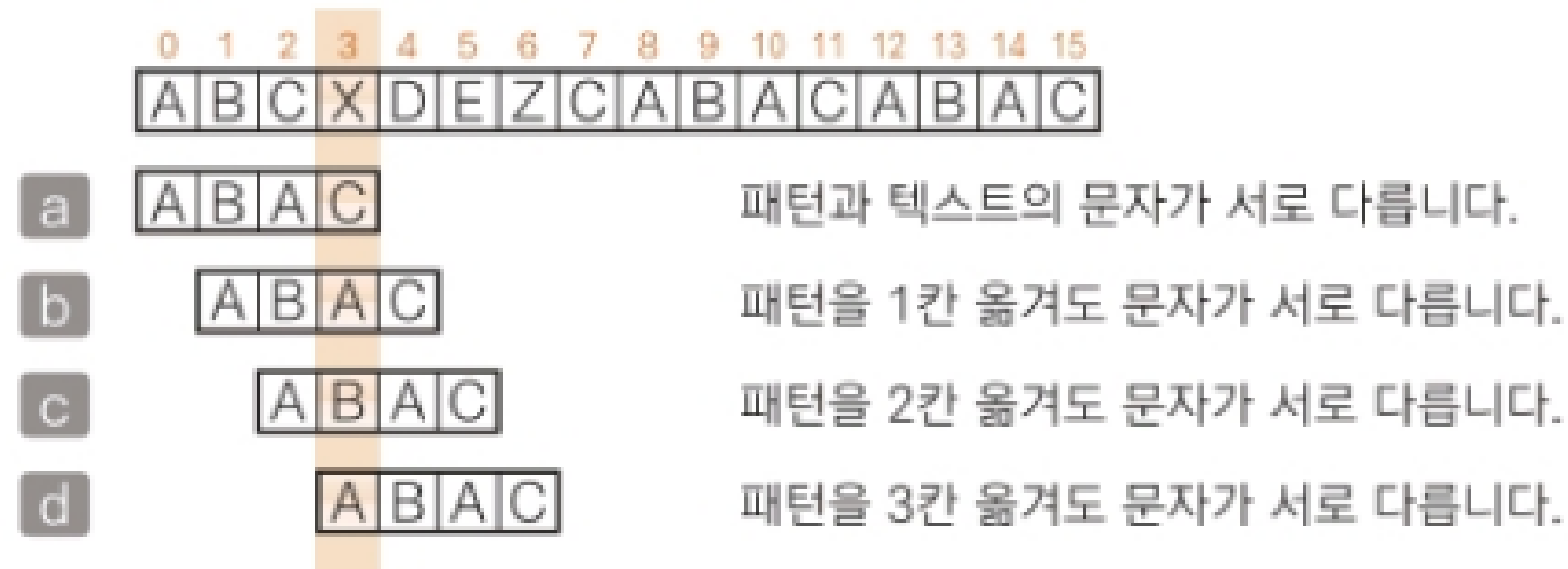
3번째 문자부터 검사를 다시 시작합니다.

05

보이어-무어 알고리즘

더 멀리 옮기는 방법을 찾자

- 문자열 비교를 오른쪽에서 왼쪽으로
- 이동은 왼쪽에서 오른쪽으로
- 패턴 오른쪽 끝의 문자와 본문 문자가 불일치하면, 패턴 길이만큼 이동
- 아래 경우에는 한번에 3칸 옮기면 됨.
- 패턴에 포함되지 않는 문자를 만난 경우:
 - 패턴 이동량이 곧 n (패턴 길이가 n 임.)
- 패턴에 포함되는 문자를 만난 경우:
 - 마지막 나오는 위치 인덱스가 k 이면 이동량은 $n-k-1$



05

보이어-무어 알고리즘

건너뛰기 표 만들기

텍스트 ... "ABCXDEZCABACABAC" 패턴 ... "ABAC"

A	B	C	D	E	F	G	H	I	J	K	L	M
1	2	4	4	4	4	4	4	4	4	4	4	4

N	O	P	Q	R	S	T	U	V	W	X	Y	Z
4	4	4	4	4	4	4	4	4	4	4	4	4

- 패턴에 없는 문자의 이동량은 모두 4
- 패턴 안에 있는 A의 이동량은 $4 - 2 - 1 = 1$

01

배운 내용 구현하기

- Brute Force법
- 라빈 카프 알고리즘
 - (단, 나누는 수를 입력받도록 하고, 패턴의 해시값과 매 탐색 시 본문 문자열의 해시값을 출력하도록 하시오.)
- KMP 알고리즘
 - (단, 경계 테이블을 출력하도록 하시오.)
- 보이어-무어 알고리즘
 - (단, 건너뛰기 표를 출력하도록 하시오.)

02

문자열 내 최소 반복 단위

문자열을 입력하면, 해당 문자열 내에서 최소 반복 단위를 구해 출력하는 프로그램을 작성하시오.
(단, KMP 알고리즘을 변형할 것.)

예)

입력: ABABABABAB 출력: AB

입력: ABCABCABC 출력: ABC

입력: ABAABA 출력: ABA

입력: ABABA 출력: ABABA

03

최장 공통 부분 구하기

두 문자열을 입력하면, 두 문자열의 공통 부분 중 가장 긴 것의 길이를 구하는 프로그램을 작성하시오.
(단, 라빈 카프 알고리즘을 변형할 것.)

예)

입력: A = "ABCDEF", B = "BCDFGH"

출력: 3 ("BCD")

입력: A = "ABAB", B = "BABA"

출력: 3 ("ABA" 또는 "BAB")

01

라빈 카프 알고리즘

해시 함수의 base

$$H[i] = S[i] * 2^{M-1} + S[i+1] * 2^{M-2} + \dots + S[i+M-2] * 2^1 + S[i+M-1] * 2^0$$

$$H[i+1] = 2 * (H[i] - C[i] * 2^{m-1}) + C[i+m+1] * 2^0$$

- 해시 함수의 base 값으로 2를 사용하기도 하지만,
- 아스키코드 문자 집합의 크기(아스키코드 상 문자의 종류 수. 128 또는 256)를 사용하기도 하고,
- 임의의 수를 사용할 수도 있다.

$$H(S) = \sum_{i=0}^{m-1} (ASCII(S[i]) \times base^{m-i-1}) \mod q$$

01

라빈 카프 알고리즘

초기 해시 값 계산

```
BASE = 256
MOD = modn

h = pow(BASE, m - 1, MOD) # h = BASE^(m-1) % MOD

pat_hash = 0
win_hash = 0
results = []

for i in range(m):
    pat_hash = (BASE * pat_hash + ord(pattern[i])) % MOD
    win_hash = (BASE * win_hash + ord(text[i])) % MOD
```

해시 값 갱신

```
if i < n - m:
    win_hash = (BASE * (win_hash - ord(text[i]) * h) + ord(text[i + m])) % MOD
    if win_hash < 0:
        win_hash += MOD
```

02

KMP 알고리즘

KMP 알고리즘의 최고 강점은, 본문을 검사하는 포인터가 절대 후퇴하지 않는다는 점.

검사한 곳을 또 검사하는 일이 없다.

경계 테이블을 활용해 몇 번째 문자부터 다시 검색할지를 결정

본문에서 한 번 검사한 문자는 다시 보지 않는다.

```
for i in range(n): # i: 본문 포인터 (뒤로 가지 않음)
    while j > 0 and text[i] != pattern[j]:
        j = btable[j - 1] # 실패 함수로 점프

    if text[i] == pattern[j]:
        j += 1

    if j == m: # 패턴 전체 일치
        results.append(i - m + 1)
        j = btable[j - 1] # 다음 탐색 위치로
```

03

보이어-무어 알고리즘

텍스트 ... "ABCXDEZCABACABAC" 패턴 ... "ABAC"

A	B	C	D	E	F	G	H	I
1	2	4	4	4	4	4	4	4

- 패턴에 포함되지 않는 문자를 만난 경우:
 - 패턴 이동량이 곧 n (패턴 길이가 n 임.)
- 패턴에 포함되는 문자를 만난 경우:
 - 마지막 나오는 위치 인덱스가 k 이면 이동량은 $n-k-1$
 - 맨 끝 문자가 패턴 내에 중복되지 않으면 이동량은 n
 - 맨 끝 문자가 패턴 내에 중복으로 있으면 이동량은 $n-k-1$ (여기서 k 는 맨 끝을 제외하고 마지막 나오는 위치).

03

보이어-무어 알고리즘

```
def boyer_moore(text, pattern):  
    n, m = len(text), len(pattern)  
    table = build_skip_table(pattern)  
    results = []  
    s = 0  
    while s <= n - m:  
        j = m - 1  
        while j >= 0 and pattern[j] == text[s+j]:  
            j -= 1  
  
        if j < 0:  
            results.append(s)  
            s += 1  
        else:  
            s += table.get(text[s+m-1], m)  
    return results
```

Q&A
